LAB 4: Implementation, Testing, and Preparation for Demo

1. OBJECTIVES

1.1 Implement your application
1.2 Design test cases using black box and white box testing techniques
1.3 Investigate coding agent capabilities
1.4 Plan your live demo
1.5 Prepare SDLC work products for delivery

2. INTRODUCTION

2.1 The design model from Lab#3 must now be implemented in a working application

2.2 The initial application skeleton needs to be further refined (i.e. some more code to be written) to accommodate additional behavior such as error handling and optimization.

2.3 The class' implementation in code should be broadly traceable to its design and requirements, but don't feel strictly bound by your previous lab submissions. You will submit updated design models reflecting your final codebase as part of Lab#5 (see the Lab#5 manual).

2.4 As the coding gets underway, the test cases are developed in parallel. The inter-mingling of codetest activities helps build confidence as working code snippets are systematically built into a sizable piece of code.

2.5 If you do not have time to implement all functionality of your design you can just implement and demonstrate the main functionality in the demo. Your design should be complete but there is no need to implement all the functionality if your schedule becomes tight.

2.6 It is now time to prepare the product for delivery and presentation to the client.

As the time allocated to the live demo is limited (15 minutes per team), prepare the material and sequence of the demo carefully, so that the salient features of your application may be highlighted to the client, amongst many competing products.

As an indication of the style the demo should take, it is good to imagine that you are presenting a prototype of a product to a customer who will choose one of the prototypes and invest in its further development for full implementation. Among several companies presenting to the customer, the customer will select the one that they consider meets their requirements and can be extended to meet their full system requirements. Meanwhile, the technical capability of the company is also an important factor to determine the winner.

3. PROCEDURE

3.1 Implement your application

3.1.1 From the design class diagram, generate the skeleton code. Implement the behavior as documented in the sequence diagrams and dialog map.

3.1.2 You may wish to organize the code into packages or folders, following the stereotypes in the design class model.

3.1.3 Document the classes and their key public methods to convey design intent and usage.

3.1.4 Use the Github repository to facilitate team collaboration.

3.2 Deploy your application for testing / design test cases using black box and white box testing techniques

3.2.1 Design test cases using equivalence class and boundary value testing techniques to test **one important control class** that implements some important application logic according to the requirements specification.

3.2.2 Design test cases using basis path testing techniques to test two methods that implement complex application logic.

3.2.3 Minimize the number of test cases through the careful selection of test input using proper black box and white box testing techniques learnt in the course.

3.2.4 Execute your test cases and document the testing results.

Note that you can develop automatic test cases using proper automatic testing framework, for example,
http://developer.android.com/training/activity-testing/activity-unit-testing.html, or
https://phpunit.de/.

Or you can perform manual testing guided by your test cases.

Test Input	Expected Output	Actual Output
…	…	…

3.3 Coding agent exercises

3.3.1 Perhaps you have already incorporated a tool such as Cline or Claude Code into your regular development workflow. In any case, these exercises are intended to ensure that every group gets some experience with agent-assisted software engineering.

Note Again, we stress that students should not pay for an AI service purely to satisfy the requirements of this course. We recommend experimenting with Cline, an in-IDE agent supported by VSCode and IntelliJ, as it has simple free configurations available with reasonable performance.

3.3.2 In 3.2.1 you identified a control class for black box testing. Once you have written some initial tests, temporarily delete the entire class (code and all!) and prompt an AI assistant of your choice to generate it as if from scratch. You can help it by providing your tests, the rest of your codebase, your design diagrams, and any other text description you feel might be helpful.

We strongly recommend that you carry out this exercise using an in-IDE agent such as Cline. When you prompt the agent to carry out the task, you can mention files in your project by name that you think it should look at. If the AI totally fails to produce anything reasonable, try experimenting with different prompts. You may also provide it with the original class file with just the function bodies deleted, to see if it is capable of completing the task with this additional structure.

3.3.3 As a group, carry out at least one further experiment of your choice using the same AI assistant on your codebase. You may brainstorm ideas among your group, or select one of the suggestions below:

- Ask the AI assistant to try and implement a feature you didn't have time to complete by hand.
- Attempt to generate the updated class diagrams required for your Lab#5 SDLC submission.
- Ask for the AI's opinion on the quality of the codebase and suggestions for refactoring. Try to get it to carry out some of its suggested changes.

3.3.4 Write two brief reflections, one for 3.3.2, and one for 3.3.3, on your experience completing each exercise. For each exercise, clearly describe how you prompted the assistant and the materials you provided, whether any iteration or extended conversion was required, what the outcome was, and whether the outcome met your expectations. If the AI was not capable of completing the task, briefly discuss whether anything could be done differently to increase the chances of success. So long as you describe the process accurately, pasting the raw input/output is not strictly required. For 3.3.3, clearly describe which experiment you chose to carry out.

3.4 Plan for live demo

3.4.1 Craft the demo script and sequence so that you maximize the opportunity to present your product to the client.

3.4.2 The client is eager to see how well the various specified functionalities can be carried out in your application. Demonstrate the various usage scenarios by the different end-users.

3.4.3 In the presentation, highlight innovative solutions in the application, as well as, elements of good software engineering practices and system design.

3.4.4 Apart from the live demo, use a screen capture tool to prepare a five-minute screen video of the application. You can provide voice-over to explain the functionality. Remember that the live demo cannot be repeated and could go wrong. The video can be seen as a second (and different) opportunity to demonstrate or demo your software which is under your control and will not go wrong.

3.5 Prepare SDLC work products for delivery

3.5.1 Re-visit the documentation from previous lab sessions to ensure that it is complete and sufficient for delivery to the client. In particular, document the traceability from requirements to design, then to implementation and test cases. See the Lab#5 manual for specific requirements.

3.5.2 The author(s) for each work product must be clearly indicated.

4. DELIVERABLES

- Working application prototype
- Source code
- Test Cases and Testing Results
- Demo script
- Your PDF report as detailed in 3.3.4.

5. REFERENCES

- Introduction to Software Engineering Design: Processes, Principles and Patterns with UML2 – C. Fox
- Object-Oriented Software Engineering: Using UML, Patterns and Java – B. Bruegge and A. Dutoit